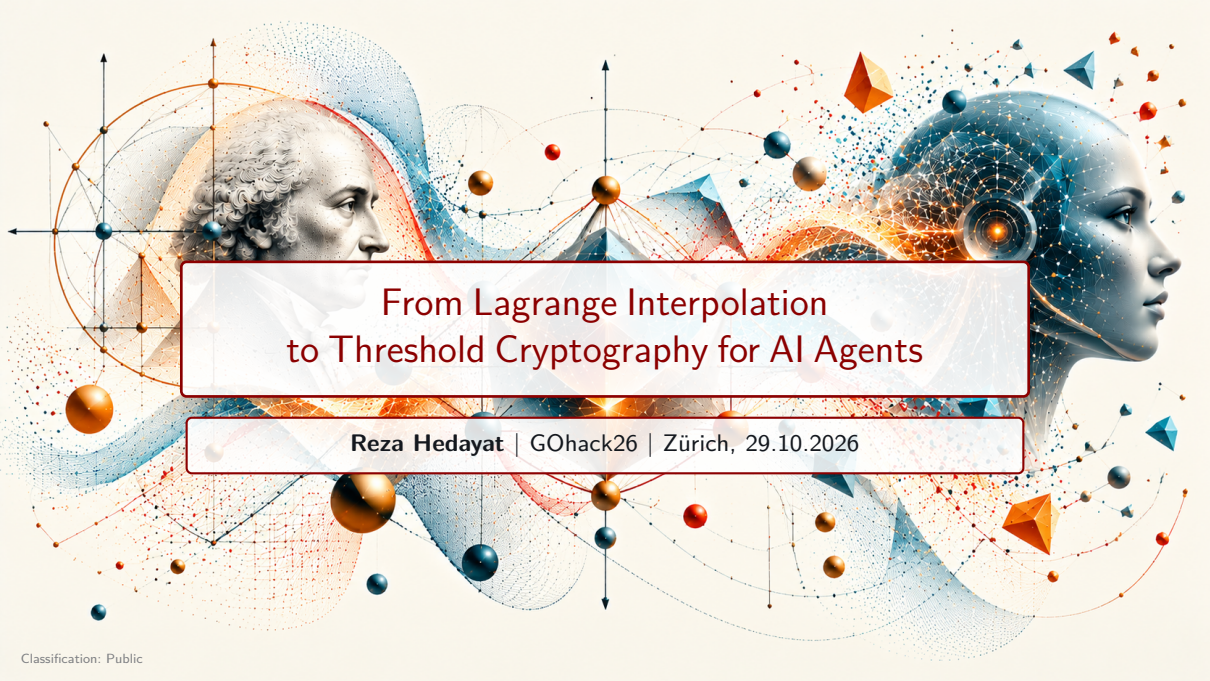


From Lagrange Interpolation to Threshold Cryptography for AI Agents

Reza Hedayat | GOhack26 | Zürich, 29.10.2026

Classification: Public

Playbook

0x01 AUTONOMY (5')
The Rise of Autonomous Agents

0x02 THREAT LANDSCAPE (10')
Threat Landscape of Agent-based Ecosystems

0x03 DISTRIBUTED TRUST (30')
From Lagrange to Distributed Post-Quantum Trust

0x04 ENDGAME (30')
Trust Under Attack

0x01

The Rise of Autonomous Agents

From generating answers to taking action



The Next Frontier of Automation

Traditional Automation

PREDEFINED PROCESSES

Explicit instructions

↓

Predefined workflow

↓

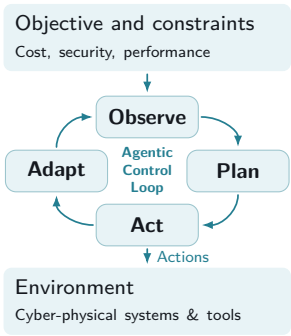
Expected output

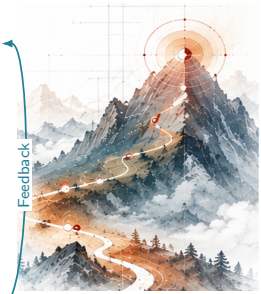


We specify **how** to perform a task.

Autonomous Agents

GOAL-DRIVEN SYSTEMS





We specify **what** to achieve.

From **executing instructions** to **pursuing objectives**.

The Autonomy–Authority Dilemma

Autonomy

WHAT MAKES AGENTS VALUABLE

- Independent decisions
- Dynamic tool selection
- Machine-speed execution
- Adaptation to new situations



Authority

WHAT MAKES AGENTS ERRATIC

- Privileged API access
- Control of assets
- Cyber-physical modifications
- Irreversible operations



Autonomy must **not** require **full authority**.

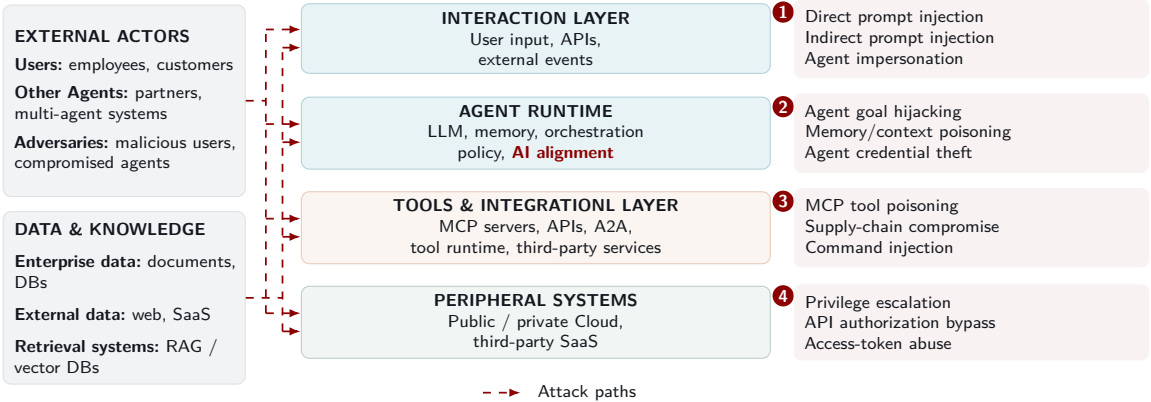
0x02

Threat Landscape

When autonomy becomes an attack surface

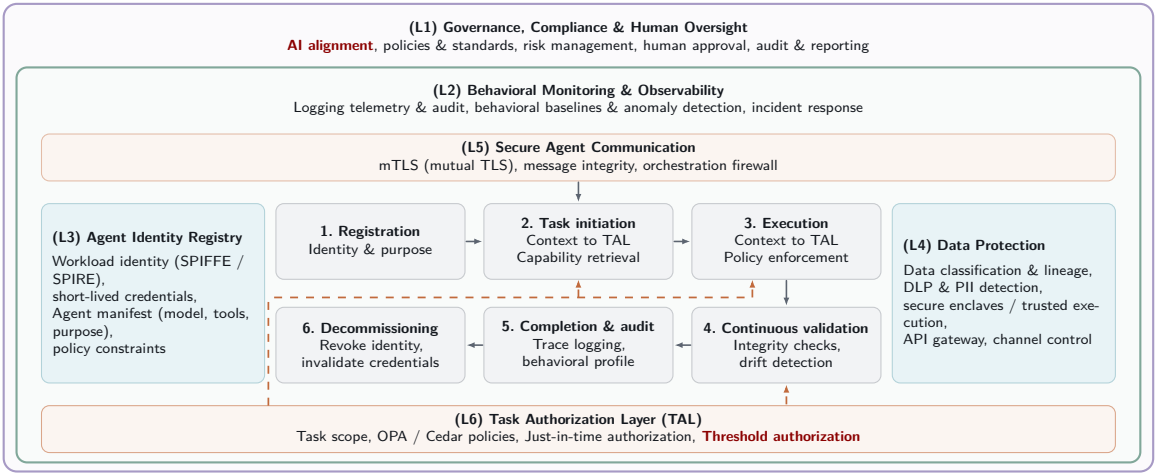


From Injection to Impact



Compromising an agent's **inputs or dependencies** is sufficient to **compromise its actions**.

Securing Agentic Systems: Zero Trust Reference Architecture*



*adapted on best-practices from Google, Microsoft & OWASP

0x03

From Lagrange to Distributed Post-Quantum Trust

Secret Sharing, Verifiable Secret Sharing & DKG

The Problem of Distributed Trust

Problem

- Suppose a cryptographic operation requires a secret $s \in \mathcal{S}$, initially held by one participant P.
- A compromise of P may give an adversary complete control over the cryptographic operation.
- How could we solve that?

Threshold access

Solution approach

- Instead, consider n participants:

$$\mathcal{P} = \{P_1, P_2, \ldots, P_n\}, n \in \mathbb{N}^+.$$

- We want to distribute cryptographic material such that a subset of participants can cooperate, while smaller subsets cannot recover the secret.
- Define a threshold access structure:

$$\Gamma_t = \{A \subseteq \mathcal{P} : |A| \geq t\},$$

  where $1 \leq t \leq n$.
- What could go wrong? Threat modelling to the rescue!

Threshold access properties

Property	Desired behavior
Correctness	Any authorized set $A \in \Gamma_t$ can reconstruct the secret or complete the designated operation.
Confidentiality	Fewer than t shares reveal no information about the secret.
Availability	The system can tolerate up to $n-t$ unavailable participants for reconstruction, assuming the remaining shares are valid.
Robustness	Malicious or inconsistent contributions are detected or handled, subject to the protocol's fault assumptions.

Secret Reconstruction vs. Distributed Cryptographic Operation

Two fundamentally **different objectives:**

Secret Reconstruction

A secret $s \in \mathcal{S}$ is distributed among n participants as shares $s_1, \ldots, s_n$.

$$\boxed{\text{Recover}(s_{i_1}, \ldots, s_{i_t}) = s}$$

An authorized subset of t participants reconstructs the original secret s.

Distributed Signing

Participants jointly compute a signature σ using their private key shares $sk_1, ..., sk_n$ to compute a partial signature $\sigma_i, i \in \mathbb{N}^+$.

$$\boxed{\text{Combine}(\sigma_{i_1}, \ldots, \sigma_{i_t}) = \sigma}$$

The resulting signature σ satisfies

$$\text{Verify}(pk, m, \sigma) = 1,$$

where the public key pk successfully verifies the signature σ for message m. The private signing key is **never reconstructed** during the protocol!

The Many Faces of Cryptographic Material

Secret material

Private keys (ECDSA, Ed25519, MLKEM, SPHINCS+, SNARKS/STARKS), symmetric keys (AES, ChaCha20), seeds.

Primary concern: confidentiality and prevention of unauthorized use.

Public material

Public keys (ECDSA, Ed25519, MLKEM, SPHINCS+, SNARKS/STARKS), certificates and public metadata (algo & key identifiers, curve, protocol version, etc.).

Primary concern: authenticity.

Derived material

Session keys (e.g. via ECDH), key-encryption keys (KEK), derived subkeys (HKDF), context-specific keys.

Primary concern: correct derivation, purpose separation and limited scope.

Distributed material

Threshold key shares (Shamir, FROST), commitments (Pedersen), distributed public keys (DKG).

Primary concern: preventing unilateral control and ensuring consistency.

Categories overlap: a secret share is both secret and distributed; a derived session key is generally secret. Cryptographic material also includes parameters, nonces, salts, IVs and commitments, with different protection requirements.

Lagrange Interpolation by Example: Construction

- Objective: Share a secret $s = 7$ among three participants ($n = 3$) such that any two ($t = 2$) can reconstruct it giving a $(2, 3)$-scheme.
- Construction: Choose a polynomial of degree one ($t - 1 = 2 - 1 = 1$):

$$f(x) = a_1 + a_2 x,$$

and set $a_0 = s = 7$ and choose $a_1 = 3$ (arbitrarily chosen) giving

$$f(x) = 7 + 3x.$$

- The secret s is the y-intercept:

$$s = f(0) = 7.$$

- Create the shares and distribute them to P_1, P_2, P_3:

$$P_i = (x_i = i, f(i)), i \in \{1, 2, 3\}.$$

Lagrange Interpolation by Example: Distributed Shares

Participant	x_i	$y_i = f(x_i)$	**Shares** (x_i, y_i)
P_1	1	10	$(1, 10)$
P_2	2	13	$(2, 13)$
P_3	3	16	$(3, 16)$

Lagrange Interpolation by Example: Reconstruction

- For example, P_1 and P_3 use their shares $(1, 10)$ and $(3, 16)$ to reconstruct the secret s.
- They calculate

$$f'(x) = a_2 = \frac{\Delta y}{\Delta x} = \frac{16 - 10}{3 - 1} = 3,$$

  and hence

$$f(1) = 10 = a_0 + 3 \implies a_0 = 7 = s,$$

  as required.
- What are the **weaknesses** in this context?

Lagrange Interpolation by Example: Weaknesses

Limitation	Explanation
No established secrecy guarantee	Without a specified random distribution for the coefficients, we cannot establish how much information a share reveals.
Numerical precision	Floating-point representation, encoding and rounding can alter shares and produce incorrect reconstruction results.
Secret reconstruction	A threshold coalition can recover the entire polynomial and generate additional shares.
No integrity verification	Malicious or corrupted shares may produce incorrect reconstruction results.
No participant authentication / authorisation	The scheme alone does not enforce which participants are authenticated / authorised.

Generalisation of Lagrange Interpolation

We reconstructed a straight line from two points. Lagrange interpolation generalizes this to any number of points:

Number of points	Polynomial	Maximum degree
2	Straight line: $f(x) = a_1 + a_2 x$	1
3	Quadratic: $f(x) = a_1 + a_2 x + a_3 x^2$	2
4	Cubic: $f(x) = a_1 + a_2 x + a_3 x^2 + a_4 x^3$	3
t	General polynomial: $f(x) = a_1 + a_2 x + a_3 x^2 + a_4 x^3 + \ldots + a_t x^{t-1}$	$t - 1$

The general formula

Given t points (x_i, y_i) with distinct x_i:

$$f(x) = \sum_{i=1}^{t} y_i \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{x - x_j}{x_i - x_j}$$

Shamir Secret Sharing (SSS) by Example

- Objective: Share a secret $s = 7$ among three participants ($n = 3$) such that any two ($t = 2$) can reconstruct it giving a $(2, 3)$-scheme.
- Construction: Choose a polynomial of degree one ($t - 1 = 2 - 1 = 1$) over the prime field $\mathbb{F}_{17}$:

$$f(x) = a_1 + a_2 x \pmod{17},$$

and set $a_0 = s = 7$ and choose $a_1 = 3$ (arbitrarily chosen) giving

$$f(x) = 7 + 3x \pmod{17}.$$

- The secret s is the y-intercept:

$$s = f(0) = 7 \pmod{17}.$$

- Create the shares and distribute them to P_1, P_2, P_3:

$$P_i = (x_i = i, f(i) \pmod{17}), i \in \{1, 2, 3\}.$$

Shamir Secret Sharing (SSS) by Example: Reconstruction

We apply the same Lagrange formula, now with finite-field arithmetic to reconstruct the secret s using the shares P_1 and P_2:

$$
\begin{aligned}
f(x) &= \sum_{i=1}^{t} y_i \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{x - x_j}{x_i - x_j} \pmod{17} \\
&= 10 \cdot (-2^{-1})(x-3) + 16 \cdot 2^{-1}(x-1) \\
&= 10 \cdot (15^{-1})(x-3) + 16 \cdot 2^{-1}(x-1) \\
&= 10 \cdot 8(x-3) + 16 \cdot 9(x-1) \\
&= 12x - 36 + 8x - 8 \\
&= 7 + 3x \pmod{17},
\end{aligned}
$$

and hence

$$
s = f(0) = 7 \pmod{17},
$$

as required.

Generalisation of SSS

1. Construct the polynomial

Given a secret $s \in \mathbb{F}_q$, choose a finite field of order

$$q = p^k > n, \qquad p \text{ prime}, \quad k \geq 1.$$

Independently sample $t-1$ uniformly random coefficients $a_2, \ldots, a_t \in \mathbb{F}_q$.

$$f(x) = s + a_2 x + \cdots + a_t x^{t-1}, \qquad a_i \xleftarrow{\$} \mathbb{F}_q$$

All arithmetic takes place in $\mathbb{F}_q$.

2. Generate and distribute the shares

Choose n distinct nonzero elements $x_1, \ldots, x_n \in \mathbb{F}_q$.

$$P_i = (x_i, f(x_i)), \qquad i = 1, \ldots, n$$

Each participant receives one share.

3. Reconstruct the secret

Any subset $I \subseteq \{1, \ldots, n\}$ with $|I| = t$ can recover the secret using Lagrange interpolation:

$$s = f(0) = \sum_{i \in I} y_i \prod_{\substack{j \in I \\ j \neq i}} \frac{-x_j}{x_i - x_j}$$

Verifiable Secret Sharing (VSS)

Problem: SSS provides secrecy, but how can a participant verify that their share is consistent with the dealer's polynomial?

Feldman's VSS

1. Generate shares using SSS

$$f(x) = s + a_1 x + \cdots + a_{t-1} x^{t-1}$$

Each participant receives a private share $y_i = f(x_i)$.

2. Publish commitments

The dealer publishes elliptic-curve points corresponding to the coefficients:

$$C_0 = sG, \qquad C_j = a_j G$$

where G is a publicly known generator of a group of order q.

3. Verify the share

Each participant checks:

$$y_i G \stackrel{?}{=} C_0 + x_i C_1 + \cdots + x_i^{t-1} C_{t-1}$$

If the equality holds, the share is consistent with the committed polynomial.

Warning: NOT PQC-safe!

Distributed Key Generation (DKG) by Example

1. Each participant generates a random polynomial

For illustration, suppose the participants independently choose:

$$\begin{aligned} P_1: &\quad f_1(x) = 2 + x \\ P_2: &\quad f_2(x) = 4 + 5x \qquad (\text{mod} 17) \\ P_3: &\quad f_3(x) = 1 + 14x \end{aligned}$$

Each participant privately sends the appropriate polynomial evaluation to every other participant, and publishes VSS commitments.

3. The resulting polynomial is our original Shamir example

$$\begin{aligned} F(x) &= f_1(x) + f_2(x) + f_3(x) \\ &= (2 + 4 + 1) + (1 + 5 + 14)x \\ &= \boxed{7 + 3x \pmod{17}}. \end{aligned}$$

2. Each participant combines the received shares

All values are computed modulo 17.

Participant	From P_1	From P_2	From P_3	Final share
P_1, $x = 1$	3	9	15	10
P_2, $x = 2$	4	14	12	13
P_3, $x = 3$	5	2	9	16

For example, P_1 computes:

$$S_1 = 3 + 9 + 15 = 27 \equiv 10 \pmod{17}.$$

Generalisation of DKG

Problem: SSS and VSS still rely on a dealer who knows the complete secret. Can we generate a shared private key without anyone ever knowing it?

How DKG works

Consider our familiar 2-of-3 threshold example over $\mathbb{F}_{17}$.

1. Each participant creates a random polynomial

$$P_1 : f_1(x) = s_1 + a_1 x$$
$$P_2 : f_2(x) = s_2 + a_2 x$$
$$P_3 : f_3(x) = s_3 + a_3 x$$

Each participant privately sends $f_i(j)$ to participant P_j, and publishes commitments for verification.

2. Each participant combines the shares received

$$\boxed{S_j = f_1(j) + f_2(j) + f_3(j) \pmod{17}}$$

Each participant now holds one share S_j of the jointly generated secret.

3. The resulting secret is distributed

$$\boxed{s = s_1 + s_2 + s_3 \pmod{17}}$$

No participant needs to reconstruct or learn s.

FROST*: Signing Without Reconstructing the Secret

After DKG, participants hold private key shares and jointly control one public key.

Our 2-of-3 example (mod 17): $P_1 : s_1 = 10$, $P_2 : s_2 = 13$, $P_3 : s_3 = 16$.

Public key: $PK = sG = 7G$, where $s = 7$ is the distributed private key and G is a public generator of a group of order 17.

Signing participants: P_1, P_3; **Lagrange weights:** $\lambda_1 = 10$, $\lambda_3 = 8$ (Lagrange coefficients at $x = 0, I = \{1, 3\}$).

1. Commit to fresh signing randomness

$$D_i = d_i G, \qquad E_i = e_i G$$

d_i, e_i: fresh secret nonces (one-time random values); D_i, E_i: public nonce commitments.

2. Compute and combine signature shares

$$R = \sum_{i \in \{1,3\}} (D_i + \rho_i E_i)$$

$$c = H(R, PK, m)$$

$$z_i = d_i + \rho_i e_i + \lambda_i s_i c$$

$$z = z_1 + z_3 \pmod{17}$$

m: message to sign; H: cryptographic hash mapped to a scalar; ρ_i: hash-derived binding factor tying signer i's nonces to the message and signing session; R: combined nonce commitment; c: challenge; z_i: partial signature; z: combined response.

3. Verify the resulting Schnorr signature

$$\sigma = (R, z), \qquad zG \stackrel{?}{=} R + cPK$$

σ: final signature. Verification uses only the message, public key and signature.

*FROST = Flexible Round-Optimized Schnorr Threshold Signatures

Post-Quantum Threshold Signatures

Scheme	Cryptographic foundation	Main difference from FROST
Mithril	Threshold ML-DSA, lattice cryptography	Uses short replicated secret shares and MPC instead of linear Schnorr signing
Quorus	Threshold ML-DSA, lattice cryptography	Uses interactive MPC and preprocessing to jointly execute ML-DSA signing
Hermine	Raccoon-style lattice signatures	Targets FROST-like functionality with short Vandermonde shares and lattice operations
Tanuki	Lattice-based threshold signatures	Targets low-round threshold signing with a different lattice-based signature construction
Vinaigrette	Multivariate cryptography (UOV/MAYO)	Replaces Schnorr's discrete-logarithm problem with multivariate polynomial problems
LEAST	Code-based group actions	Uses linear-equivalence actions rather than elliptic-curve scalar multiplication
PQarrots / Macaw	Isogeny-based group actions	Uses group-action constructions instead of Schnorr's elliptic-curve discrete logarithm
Haystack	Stateful hash-based signatures	Relies on hash-based security and requires management of signing state

Distributed Trust: What Have We Actually Secured?

Challenge	What we have learned
Single point of compromise	Shamir distributes secrets; DKG removes the need for a trusted key-generating dealer.
Malicious participants	VSS verifies share consistency, but cannot guarantee honest decisions.
Key exposure	FROST enables signing without reconstructing the private key during normal operation.
Authorization	A threshold counts participating signers; it does not inherently enforce business policies.
Independence	Participants must have genuinely separate control, infrastructure and security boundaries.
Collusion and availability	Threshold selection balances resistance to compromise against operational resilience.

What This Means for Autonomous Agents

An autonomous agent should not be granted unrestricted signing authority. Instead, it should operate within an architecture where:

- Authorization policies are explicitly defined and independently enforced.
- Critical actions require appropriate human involvement.
- Cryptographic keys remain distributed, with no reconstruction during routine operations.
- Signers verify the exact action they authorize, rather than blindly approving an agent's request.

Final takeaway

Distributed cryptography reduces dependence on individual system components. It does not eliminate the need to trust, verify and govern the system as a whole.



0x04

Endgame

Trust Under Attack


Julia
Capital
FORGET SWAROVSKI.
THINK CRYPTOSTEEL.
Top 3 quiz scorers win a
Cryptosteel.
Less sparkle. More resilience.
1 BITCOIN
2 FREEDOM
3 SELFCUSTODY
CRYPTOSTEEL

Final Quiz: Trust Under Attack



Good luck!